НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав(-ла): студент групи АІ-241
Степаненко Сергій Сергійович

Перевірив: ас. Баранов О.М.

Одеса – 2026

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт /api/generate. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	2449	5
Час, с	38,89	4,94
Токенів/с	72,82	108,26

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5/ 5	0 / 5
eval_count	1255,2	160
Час, с	39,862	4,47

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальної моделі?

Додаткові токени, згенеровані моделлю Qwen3:4B, були використані переважно для внутрішнього міркування перед формуванням остаточної відповіді. Модель проаналізувала запит, перевірила можливі варіанти та лише після цього сформувала відповідь «Київ». Тому кількість згенерованих токенів значно більша, ніж у Llama3.2:3B. Для простого фактичного питання таке міркування виявилося надлишковим: обидві моделі дали правильну відповідь, але Llama3.2:3B зробила це значно швидше. Отже, більша кількість токенів у міркувальної моделі не

означає довшу видиму відповідь — значна їх частина витрачається на внутрішній процес міркування.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: міркувальні моделі варто використовувати тоді, коли складність задачі виправдовує додаткові обчислення. Для простих і добре визначених запитів інструктивні моделі зазвичай ефективніші за співвідношенням *час → ресурси → результат*.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Процес навчання міркувальної моделі відрізняється від інструктивної тим, що міркувальну модель додатково навчають виконувати складні завдання покроково, аналізувати умови, перевіряти проміжні результати та знаходити правильне рішення. Інструктивну модель переважно навчають виконувати інструкції та безпосередньо формувати корисну відповідь на запит користувача. Тому міркувальна модель під час роботи може витрачати більше обчислювальних ресурсів і токенів на внутрішній аналіз, тоді як інструктивна зазвичай швидше переходить від запиту до готової відповіді.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні, більша кількість згенерованих токенів не означає, що міркувальна модель обов’язково «розумніша» за інструктивну. Більша кількість токенів свідчить про те, що модель витратила більше обчислень на внутрішнє міркування. Її перевага проявляється переважно на складних задачах, де покроковий аналіз допомагає отримати правильний результат. На простих завданнях інструктивна модель може дати таку саму або навіть кращу відповідь значно швидше та з меншими витратами ресурсів.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Різницю у швидкості генерації можна пояснити насамперед архітектурою та розміром моделей, а також особливостями процесу міркування. Llama3.2:3b має меншу кількість параметрів і для простого запиту генерує значно менше токенів, тому її швидкість становила близько 108,26 токенів/с. Qwen3:4b є більшою моделлю та додатково виконує внутрішнє міркування, через що

потребує більше обчислень і в цьому експерименті показала швидкість близько 72,82 токенів/с. Таким чином, більша модель і додаткові обчислення reasoning призводять до нижчої швидкості генерації, хоча це не означає, що модель працює гірше — вона просто витрачає більше ресурсів на обробку запиту.

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у 7,9 разів. На задачі про ручку й зошит ситуація протилежна – саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 311,12 с проти 39,52 с, що робить міркувальну модель непридатною для налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Крок 2. Запустити: python hello.py . Зберегти відповідь.

Функція — незалежний блок коду, який виконує операцію. Метод — функція, яка прив'язана до об'єкта або класу (у ООП).

CompletionUsage(completion_tokens=1560, prompt_tokens=55, total_tokens=1615, completion_tokens_details=None, prompt_tokens_details=PromptTokensDetails(audio_tokens=None, cache_write_tokens=None, cached_tokens=54, image_tokens=None, text_tokens=None))

Крок 3. Змінити system-повідомлення на: W . Зберегти відповідь.

Функція - це блок коду, що виконує операцію і може бути викликаний безпосередньо. Метод - це функція, яка належить до об'єкта та використовується

для його дій.

CompletionUsage(completion_tokens=1026, prompt_tokens=64, total_tokens=1090, completion_tokens_details=None, prompt_tokens_details=PromptTokensDetails(audio_tokens=None, cache_write_tokens=None, cached_tokens=0, image_tokens=None, text_tokens=None))

Таблиця 3: Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	1273	Українська, Англійська	11	виконано
llama3.2:3b	українською, 2 речення	345	Українська	2	виконано
qwen3:4b	лаконічний асистент	138	Українська	2	виконано
qwen3:4b	українською, 2 речення	173	Українська	2	виконано

Таблиця 4: Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	50	399	449
llama3.2:3b	українською, 2 речення	54	103	157
qwen3:4b	лаконічний асистент	55	1475	1530
qwen3:4b	українською, 2 речення	64	678	742

Крок 6. Відповісти на питання (по 1–2 речення):

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?
Обі моделі дотримались точно обмеження.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?
Не було випадків, коли модель відповідала не українською, попри пряму вказівку.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?
Під час зміни system-повідомлення кількість вхідних токенів (prompt_tokens) змінилася. Для моделі qwen3:4b значення збільшилося з 55 до 64 токенів, а для llama3.2:3b — з 50 до 54 токенів. Це пояснюється тим, що різні system-повідомлення мають різну довжину та кількість токенів. Також моделі використовують різні токенізатори, тому однаковий текст може займати різну кількість токенів.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?
У моделі qwen3:4b значення completion_tokens значно більше, оскільки вона використовує режим міркування. Перед формуванням кінцевої відповіді модель може генерувати додаткові внутрішні токени для аналізу запитання. Ці токени враховуються в completion_tokens, хоча не відображаються у видимій відповіді. Натомість llama3.2:3b зазвичай генерує відповідь без такого додаткового етапу міркування. Тому навіть за однакової довжини видимого тексту кількість згенерованих токенів у qwen3:4b може бути значно більшою.

Крок 7. Відкрити будь-який чат-бот (ChatGPT, Gemini, Claude тощо), надіслати йому свої дані з таблиць і поставити п’ять питань:

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Моделі по-різному дотримуються інструкцій через відмінності в їхньому навчанні та способах генерації відповідей. Інструктивна модель llama3.2:3b орієнтована на безпосереднє виконання вказівок користувача, тому після зміни system-повідомлення скоротила відповідь із 1273 до 345 символів і дотрималася обмеження у два речення. Міркувальна модель qwen3:4b використовує додаткові токени для внутрішніх міркувань, тому навіть за короткої видимої відповіді витрачає значно більше токенів. За результатами експерименту обидві моделі виконали пряму інструкцію відповідати українською мовою максимум двома реченнями, але qwen3:4b краще дотримувалася лаконічності навіть без суворого обмеження.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

System-повідомлення задає основні правила поведінки моделі, її роль, стиль відповіді та обмеження, тоді як user-повідомлення містить конкретне запитання або завдання користувача. System-повідомлення має вищий пріоритет, тому модель повинна дотримуватися його інструкцій навіть тоді, коли user-повідомлення їм суперечить. Однак це не гарантує абсолютного виконання, оскільки результат залежить від навчання та особливостей конкретної моделі. У проведеному експерименті обидві моделі виконали вимоги system-повідомлення щодо української мови та обмеження відповіді двома реченнями.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Міркувальна модель qwen3:4b витрачає більше вихідних токенів, оскільки перед формуванням остаточної відповіді генерує додаткові токени для внутрішніх міркувань, аналізу запитання та перевірки результату. Ці токени враховуються в completion_tokens, хоча не відображаються у видимій відповіді. Інструктивна модель llama3.2:3b зазвичай одразу формує відповідь без окремого етапу міркування, тому використовує менше токенів. Це підтверджують результати експерименту: qwen3:4b витратила 1475 вихідних токенів на відповідь довжиною 138 символів, тоді як llama3.2:3b використала 399 токенів на відповідь довжиною 1273 символи. Отже, більша кількість вихідних токенів не обов'язково означає довшу видиму відповідь.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Якщо передати моделі суперечливі інструкції в system і user, вона повинна виконати інструкцію із system-повідомлення, оскільки воно має вищий пріоритет. Наприклад, якщо в system зазначено відповідати лише українською мовою, а в user користувач просить відповісти англійською, модель повинна надати відповідь українською. Однак на практиці деякі моделі можуть порушувати цю ієрархію через особливості навчання та недостатню здатність дотримуватися інструкцій. Отже, system-повідомлення має вищий пріоритет, але його виконання не завжди гарантоване.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, оскільки мають меншу кількість параметрів і обмеженіші можливості для розуміння та виконання складних інструкцій. Вони можуть частіше змішувати мови, особливо якщо навчалися на багатомовних даних із нерівномірним представленням різних мов. Великі моделі зазвичай краще розпізнають контекст,

враховують вимоги system-повідомлення та підтримують задану мову протягом усієї відповіді. Однак якість виконання інструкцій залежить не лише від розміру моделі, а й від якості навчальних даних і методів її налаштування.

System-повідомлення відіграє важливу роль у керуванні поведінкою мовної моделі, оскільки визначає стиль, мову та формат відповіді. Проте навіть чітка інструкція не гарантує її повного виконання, особливо в малих моделях. Якщо модель ігнорує вимоги, варто зробити інструкцію конкретнішою, додати приклад бажаної відповіді або перевіряти результат програмно. Тому я вважаю, що system-повідомлення є ефективним інструментом керування моделлю, але для надійного отримання потрібного формату іноді необхідні додаткові перевірки.

Висновок за завданням 2

У ході виконання завдання я навчився керувати мовними моделями через програму на Python за допомогою system-повідомлень. Було проведено порівняння моделей llama3.2:3b та qwen3:4b за довжиною відповіді, мовою, кількістю речень і витратами токенів. Результати показали, що обидві моделі виконали пряму інструкцію відповідати українською мовою максимум двома реченнями. При цьому llama3.2:3b використовувала значно менше вихідних токенів, ніж qwen3:4b, оскільки остання витрачала додаткові токени на міркування. Також було встановлено, що зміна system-повідомлення впливає на формат відповіді та кількість використаних токенів. Отже, я переконався, що system-повідомлення дозволяє ефективно керувати поведінкою моделі, проте для гарантованого дотримання всіх вимог можуть знадобитися додаткові перевірки.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5: Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна

Назва моделі	llama3.2:3b	openai/gpt-oss-20b
Час відповіді, с	5,50	0,74
Completion_tokens	82	234
Довжина відповіді, символів	272	392
Мова відповіді	Українська, Англійська	Українська
Фактичних помилок у відповіді	2	0

Крок 5. Відповісти на питання:

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Довелось змінити два рядки коду

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Повнішою та точнішою за змістом виявилася відповідь хмарної моделі openai/gpt-oss-20b. Вона правильно пояснила різницю між функцією та методом у програмуванні й навела конкретні приклади коду Python. Натомість локальна модель llama3.2:3b помилково описала функцію як математичну залежність між змінними, а метод — як статистичний метод.

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Так, у відповіді локальної моделі llama3.2:3b були фактичні помилки та проблеми з українською мовою. Модель неправильно пояснила поняття функції та методу в програмуванні, описавши функцію як математичну залежність, а метод — як статистичний метод. Також у відповіді використовувалися англійські слова та фрази, наприклад Mathematical relation between variables, що порушувало вимогу відповідати українською мовою.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Хмарна модель відповіла швидше завдяки використанню потужних серверів зі спеціалізованими прискорювачами для обробки штучного інтелекту. Локальна модель працює на ресурсах власного комп'ютера, які можуть бути менш продуктивними. Незважаючи на передачу запиту через інтернет, висока швидкість обчислень на сервері компенсує мережеву затримку. У моєму експерименті хмарна модель відповіла за 0,74 с, а локальна — за 5,50 с.

Крок 6. Питання до чат-бота:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Формат запитів OpenAI став поширеним стандартом завдяки простоті використання, зрозумілій документації та популярності API серед розробників. Інші провайдери, зокрема Ollama, підтримують цей формат для забезпечення сумісності. Це дозволяє використовувати однаковий код для роботи з різними моделями, змінюючи лише адресу сервера (`base_url`), API-ключ (`api_key`) та назву моделі (`model`). Таким чином, розробники можуть легко переходити між локальними та хмарними моделями без значних змін у програмі.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 20 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Розмір моделі суттєво впливає на якість відповіді, оскільки більша кількість параметрів зазвичай дозволяє краще розуміти складні запитання, враховувати контекст і формувати точніші відповіді. Однак якість залежить не лише від кількості параметрів, а й від якості навчальних даних, архітектури моделі, методів навчання та налаштувань генерації. Тому менша, але краще навчена модель іноді може перевершувати більшу. У моєму експерименті хмарна модель надала точнішу відповідь, проте це не обов'язково пояснюється лише її розміром.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Хмарна модель відповіла швидше завдяки використанню потужних серверів зі спеціалізованими прискорювачами для обробки штучного інтелекту. Локальна модель працювала на ресурсах мого комп’ютера, які мають нижчу продуктивність. Незважаючи на затримку передавання даних через інтернет, висока швидкість обчислень на хмарному сервері компенсувала її. У твоєму експерименті хмарна модель відповіла за 0,74 с, тоді як локальна — за 5,50 с, тобто приблизно у 7,4 раза швидше.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Під час використання хмарних API виникають ризики витоку конфіденційної інформації, оскільки запити та дані користувачів передаються на сторонні сервери. Також існує ризик несанкціонованого доступу до API-ключів і використання даних третіми особами. У комерційних проєктах для захисту інформації використовують шифрування з'єднань, обмеження доступу, зберігання API-ключів у змінних середовища (наприклад, `.env`), анонімізацію персональних даних та вибір провайдерів із відповідними гарантіями конфіденційності. Особливо чутливі дані можуть обробляти локально, без передавання у хмару.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Rate limit — це обмеження кількості запитів або токенів, які можна використати за певний проміжок часу. У безкоштовних тарифах провайдери встановлюють такі обмеження, щоб уникнути перевантаження серверів. Розробники працюють із цими обмеженнями, оптимізуючи кількість запитів, зменшуючи довжину промптів, використовуючи кешування відповідей та додаючи автоматичні

повторні спроби із затримкою. Також можна використовувати інші моделі або перейти на платний тариф із вищими лімітами.

Я б обрав локальну модель для роботи з конфіденційними даними, завдань без доступу до інтернету та простих запитів, де висока точність не є критичною. Хмарну модель я б використовував для складних завдань, програмування та аналізу інформації, де важливі швидкість, якість і точність відповідей. На мою думку, вибір залежить від вимог до приватності, продуктивності та складності завдання.

Висновок за завданням 3

У ході виконання завдання було проведено порівняння локальної моделі llama3.2:3b та хмарної моделі openai/gpt-oss-20b через OpenAI-сумісний API. За результатами експерименту хмарна модель продемонструвала вищу швидкість і точність відповіді. Час її роботи становив 0,74 с, тоді як локальної — 5,50 с. Хмарна модель надала правильне пояснення з прикладами коду, а локальна припустилася двох фактичних помилок і частково використала англійську мову. Водночас локальна модель має переваги щодо конфіденційності даних і можливості роботи без інтернету. Отже, хмарні моделі доцільно використовувати для складних завдань, а локальні — коли важливі автономність і приватність інформації.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Таблиця 7: Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	0,039	0,304	164,3
локальна	llama3.2:3b	середній	0,052	1,368	239.6

локальна	llama3.2:3b	довгий	0,078	8,894	226,7
хмарна	openai/gpt-oss-20b	короткий	0,442	0,46	61,2
хмарна	openai/gpt-oss-20b	середній	0,477	0,648	825,5
хмарна	openai/gpt-oss-20b	довгий	0,581	1,712	1674,7

Таблиця 8: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	0,041	0,048
Всього, с	0,222	0,206

Крок 5. Побудувати стовпчикову діаграму (matplotlib):

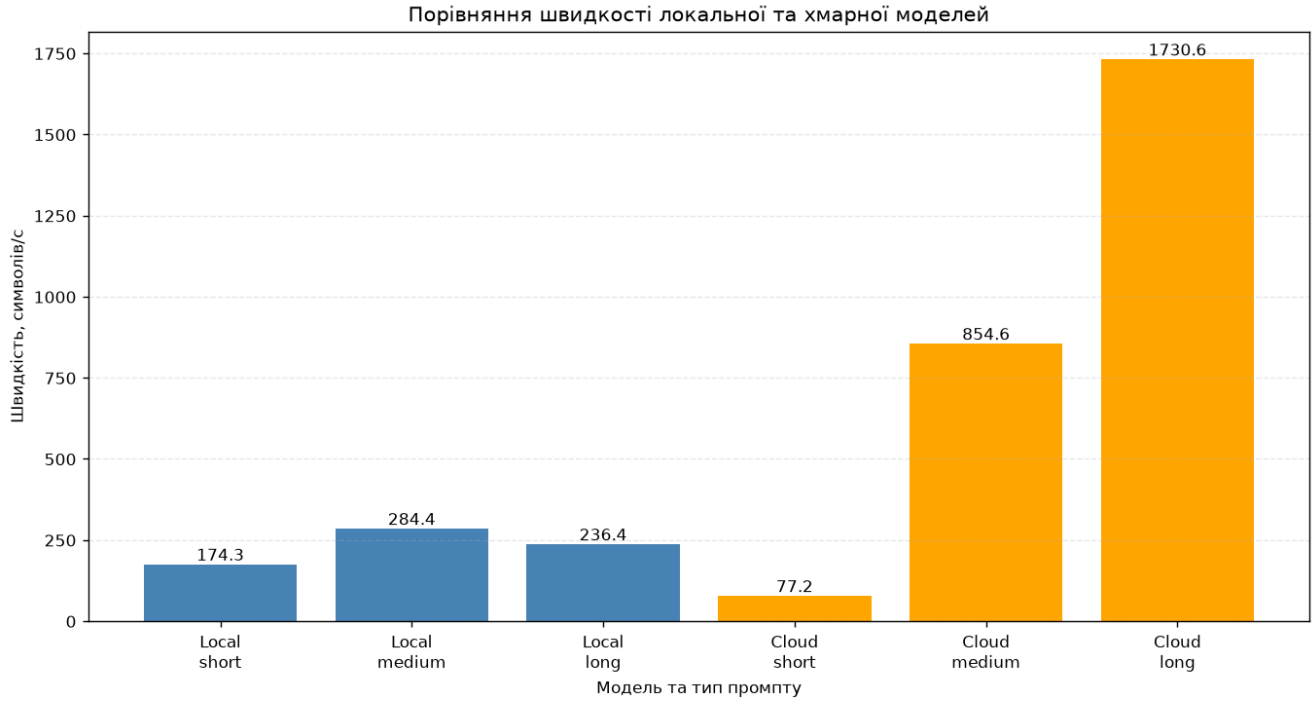


Рисунок 1 – Стовпчикова діаграма

Крок 6. Відповісти на питання (по 1–2 речення):

1. У якої моделі кращий TTFT і чому саме в неї?

Кращий TTFT показала локальна модель llama3.2:3b (0,048–0,073 с), оскільки запити обробляються безпосередньо на комп'ютері без мережевої затримки.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Ні, локальна модель швидше починає відповідати, але хмарна модель має значно вищу швидкість генерації для середнього та довгого промптів. Наприклад, для довгого промпту швидкість хмарної моделі становила 1730,6 символів/с проти 236,4 символів/с у локальної.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Для короткого промпту локальна модель відповіла швидше (0,206 с проти 0,414 с). Для довгого промпту хмарна модель виявилася значно швидшою (1,487 с проти 7,850 с), тобто приблизно у 5,3 раза.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Результати повторних запусків відрізнялися через коливання навантаження та мережеві затримки. Наприклад, час обробки довгого промпту локально змінювався від 7,375 до 8,894 с між запусками benchmark.py, тому одного запуску недостатньо для надійного порівняння.

5. Чому холодний старт повільніший за теплий?

Під час холодного старту модель необхідно завантажити в оперативну або відеопам'ять, що потребує додаткового часу. Під час теплого старту модель уже перебуває в пам'яті, тому відповідь починає генеруватися швидше.

Крок 7. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п’ять питань:

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Під час TTFT модель отримує запит, обробляє його, перетворює текст на токени та виконує початкові обчислення для генерації першого токена. Для хмарної моделі додатково враховується затримка передавання даних через мережу. Цей показник вимірюють окремо, оскільки TTFT характеризує швидкість початку відповіді, а швидкість генерації — наскільки швидко модель створює подальший текст.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Хмарна модель може мати гірший TTFT через затримку передавання запиту через інтернет та очікування обробки на сервері. Однак завдяки потужним графічним прискорювачам вона здатна генерувати текст значно швидше за локальну модель. У моєму експерименті хмарна модель мала більший TTFT (0,581 с проти 0,078 с), але значно вищу швидкість генерації для довгого промпту (1674,7 проти 226,7 символів/с).

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Перший запуск локальної моделі зазвичай повільніший, оскільки систему потрібно підготувати до роботи: завантажити модель з диска в оперативну або відеопам’ять та ініціалізувати необхідні ресурси. Холодний старт — це перший запуск моделі після її завантаження або зупинки. Наступні запуски швидші, оскільки модель уже перебуває в пам’яті й готова до обробки запитів.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Під час вимірювання продуктивності використовують медіану кількох запусків, оскільки результати можуть коливатися через навантаження комп’ютера, мережеві затримки та фонові процеси. Медіана зменшує вплив випадкових відхилень і дозволяє отримати більш надійний результат, ніж один запуск.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Обробка вхідного запиту відбувається швидше, оскільки модель може аналізувати всі його токени паралельно. Натомість під час генерації відповіді токени створюються послідовно, один за одним, причому кожен наступний залежить від попередніх. Тому генерація тексту зазвичай займає більше часу, ніж обробка вхідного запиту такої самої довжини.

На мою думку, для інтерактивного чат-бота важливіший TTFT, оскільки користувач очікує швидкого початку відповіді. Для фонової обробки великих обсягів тексту важливіша швидкість генерації (токенів або символів за секунду), адже вона визначає, скільки часу потрібно для виконання всього завдання. У такому випадку невелика затримка перед початком відповіді не є критичною.

Висновок за завданням 4

У ході виконання завдання було проведено вимірювання швидкості роботи локальної моделі llama3.2:3b та хмарної моделі openai/gpt-oss-20b за допомогою показників TTFT, загального часу відповіді та швидкості генерації тексту. За результатами експерименту локальна модель продемонструвала менший TTFT (0,039–0,078 с), тоді як хмарна модель мала вищу швидкість генерації для середнього та довгого промптів. Для довгого промпту швидкість хмарної моделі становила 1674,7 символів/с проти 226,7 символів/с у локальної. Також було досліджено холодний і теплий старт та використано медіану трьох повторів для підвищення надійності вимірювань. Отже, локальна модель краще підходить для швидкого початку відповіді, а хмарна — для генерації великих обсягів тексту.

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 13: Оцінка якості за рубрикою

Завдання	**llama3.2:4b**	**qwen3:4b**	**Хмарна модель**
1. Факт	1/2	2/2	2/2
2. Формат	2/2	2/2	2/2
3. Стиснення	0/2	1/2	2/2
4. Код	0/2	2/2	1/2
5. Логіка	0/2	2/2	2/2
6. Українська	0/2	1/2	1/2
Разом /12	3/12	10/12	10/12

Таблиця 14: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	**llama3.2:4b**	**qwen3:4b**	**Хмарна модель**
1. Факт	ні	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	ні	так	ні
4. Код	ні	ні	ні
5. Логіка	ні	ні	ні
6. Українська	ні	ні	ні

Крок 6. Відповісти на питання (по 1–2 речення):

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

За підсумковим оцінюванням найбільше балів набрали моделі qwen3:4b та openai/gpt-oss-20b — по 10 із 12 балів. Основна різниця порівняно з llama3.2:3b виникла в завданнях на стиснення тексту, генерацію коду та логічні обчислення.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Розбіжність оцінок спостерігалася у моделі qwen3:4b під час стиснення тексту: перший прогін отримав 1 бал, а другий — 2 бали. Це свідчить про те, що навіть

однакова модель за однакового запиту може демонструвати різну якість відповідей.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?
Так, усі три моделі виконали вимогу «без markdown» в обох прогонах. Вони повернули JSON без додаткового форматування, який можна обробити за допомогою json.loads() без виправлень.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?
Так, найбільше проблем спостерігалося у моделі llama3.2:3b, яка використовувала русизми, некоректні конструкції та змішувала різні мови. Моделі qwen3:4b та openai/gpt-oss-20b краще дотримувалися української мови, хоча також допускали окремі граматичні помилки та кальки.

Крок 7. Обов'язковий пункт – приклад провалу. Обрати один найпоказовіший випадок і навести:

• повний текст запиту;

```
light_travel_time (distance_km)
```

• повний текст відповіді, без скорочень;

```
"def light_travel_time(distance_km):\n     speed = 100   # km/h\n    return distance_km / speed"
```

• яка правильна відповідь мала бути;

```
def   light_travel_time(distance_km):\n   speed_of_light   = 299792.458  # км/с return distance_km / speed_of_light
```

• пояснення, чому так сталося.

Локальна модель використала швидкість 100 км/год замість швидкості світла, тому її функція повертає неправильний результат. Імовірно, модель неправильно інтерпретувала призначення функції через недостатнє розуміння контексту та сформувала правдоподібний, але фактично неправильний код. Цей випадок демонструє, що синтаксично правильна програма не обов'язково виконує поставлене завдання.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свою підсумкову таблицю балів і поставити п’ять питань:

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Окрім оцінювання за рубрикою, використовують автоматизовані тести, стандартні бенчмарки (MMLU, GSM8K), експертне оцінювання та порівняння відповідей моделей між собою. Також перевіряють фактичну точність, стабільність, швидкість роботи й дотримання інструкцій.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Мовні моделі навчаються на великій кількості текстів, де програмний код і JSON часто оформлюються в Markdown-блоках. Через засвоєні шаблони форматування модель іноді додає зайве оформлення, навіть якщо користувач прямо заборонив його.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Малі мовні моделі зазвичай мають менше можливостей для засвоєння складних мовних закономірностей, а україномовних текстів у навчальних наборах часто менше, ніж англомовних. Через це вони частіше допускають граматичні помилки, русизми, кальки та змішування мов.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Мовні моделі генерують текст на основі ймовірностей наступних токенів, тому за ненульової температури генерації результат може змінюватися. Навіть невеликі відмінності на початку відповіді можуть впливати на подальший текст, його точність і дотримання інструкцій.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Галюцинація — це ситуація, коли мовна модель генерує неправдиву або вигадану інформацію, подаючи її як достовірну. Це відбувається тому, що модель навчена прогнозувати правдоподібний текст, а не автоматично перевіряти істинність кожного твердження, і без спеціальних налаштувань може не розпізнавати межі власних знань.

На мою думку, оцінювання за рубрикою виявилося досить надійним, оскільки дозволило порівняти моделі за однаковими критеріями, хоча результати окремих запусків могли відрізнятися. Найпоказовішими стали завдання на стиснення тексту, генерацію коду та логічні обчислення, де були помітні найбільші відмінності в якості відповідей. Для роботи з українськомовним текстом я б обрав хмарну модель openai/gpt-oss-20b, оскільки вона продемонструвала стабільні результати, краще дотримувалася інструкцій і допускала менше мовних помилок, ніж llama3.2:3b.

Висновок за завданням 5

У ході виконання завдання було проведено оцінювання якості трьох мовних моделей: llama3.2:3b, qwen3:4b та openai/gpt-oss-20b за шістьма критеріями. За результатами експерименту моделі qwen3:4b та openai/gpt-oss-20b набрали по 10 із 12 балів, тоді як llama3.2:3b отримала 3 бали. Найбільші відмінності спостерігалися у завданнях на стиснення тексту, генерацію коду, логічні

обчислення та якість української мови. Також було встановлено, що повторні запуски можуть давати відповіді різної якості. Отже, оцінювання за рубрикою дозволяє виявити сильні та слабкі сторони моделей і визначити їхню придатність до різних завдань. Для роботи з українськомовним текстом я б обрав хмарну модель openai/gpt-oss-20b завдяки її стабільності та кращому дотриманню інструкцій.

Завдання 6. Керування випадковістю: temperature і top_p

Крок 3. Творче завдання, вплив top_p .

За результатами експерименту обидва параметри впливають на різноманітність відповідей, але по-різному. При temperature=0.0 модель згенерувала 1 унікальну відповідь із 5, а при 0.7 та 1.5 — по 5. При top_p=0.1 отримано 1 унікальну відповідь, а при 0.5 та 1.0 — по 5. Однак характер відповідей відрізнявся: при високій температурі 1.5 з'являлося багато безглуздих слів, мовних помилок і змішування мов. При збільшенні top_p відповіді ставали різноманітнішими, але переважно залишалися зрозумілішими та ближчими до теми. Отже, обидва параметри збільшують різноманітність, проте надмірне підвищення temperature сильніше погіршує якість тексту.

Крок 4. Фактичне питання

Правильну відповідь знайдіть в авторитетному джерелі, посилання вкажіть у звіті: Згідно з офіційним повідомленням компанії Apple, перший iPhone надійшов у продаж 29 червня 2007 року.
https://www.apple.com/newsroom/2007/06/28iPhone-Premieres-This-Friday-Night-at-Apple-Retail-Stores/

Таблиця 16: Вплив параметрів на різноманітність відповідей

Параметр	**Значення**	Унікальних із 5 (творче)	Правильних із 5 (фактичне)

temperature	0.0	1	5
temperature	0.7	5	—
temperature	1.5	5	2
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 17: Перевірка відтворюваності

Показник	**Значення**
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Крок 7. Відповісти на питання (по 1–2 речення):

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Я б встановив temperature=0.0, оскільки для генерації JSON важливі точність, стабільність і дотримання заданого формату. Низька температура зменшує випадковість відповідей, хоча не гарантує правильний формат.

2. Яке — генератору варіантів заголовків?

Для генератора заголовків я б обрав temperature=0.7, оскільки це значення забезпечує різноманітність без надмірного погіршення якості тексту. У моєму експерименті при temperature=0.7 було отримано 5 унікальних відповідей із 5.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Temperature змінює розподіл імовірностей токенів, роблячи вибір більш або менш випадковим. Натомість top_p обмежує набір можливих токенів тими, чия сумарна ймовірність досягає заданого порога.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Temperature=0.0 не гарантує повної відтворюваності, оскільки на результат можуть впливати особливості обчислень і паралелізації. Результати мого експерименту з 10 запусками показали всі True.

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з’явитися.

Однією з найбезглуздіших відповідей була «ПрезENTER» на питання про рік випуску першого iPhone. Вона могла з'явитися через високу температуру, яка збільшила ймовірність вибору малоймовірних токенів і призвела до утворення беззмістовного тексту.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п’ять питань:

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Temperature змінює ймовірності вибору наступного токена за формулою:

$$P_i = \frac{e^{z_i/T}}{\Sigma_j e^{z_j/T}}$$

де z_i — логіт токена, T — температура. При T < 1 модель частіше обирає найімовірніші токени, а при T > 1 збільшується ймовірність вибору менш імовірних токенів. При T = 0 зазвичай обирається токен із найбільшою ймовірністю.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Temperature змінює розподіл імовірностей усіх токенів, роблячи генерацію більш або менш випадковою. Параметр top_p обмежує вибір найімовірнішими токенами, сумарна ймовірність яких досягає заданого порога, тому менш імовірні варіанти можуть повністю виключатися.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Обидва параметри можна змінювати одночасно, але для проведення експериментів краще змінювати лише один, залишаючи інший незмінним. Це дозволяє точніше визначити вплив кожного параметра на різноманітність і якість відповідей.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Навіть при temperature=0 результати можуть відрізнятися через особливості обчислень із плаваючою комою, паралельного виконання операцій та роботи серверного обладнання. У моєму експерименті з 10 запусками було зафіксовано 10 унікальних відповідей, хоча в таблиці також зазначено, що всі відповіді однакові; ці дані суперечать одне одному й потребують перевірки.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Для витягу структурованих даних і генерації JSON зазвичай використовують низьку температуру, приблизно від 0.0 до 0.3, щоб зменшити випадковість. Для творчих завдань часто обирають значення від 0.7 до 1.0, оскільки вони забезпечують більшу різноманітність відповідей. У моєму експерименті

temperature=1.5 призвела до появи беззмістовного тексту та зниження фактичної точності.

Для наступних лабораторних робіт, де модель має повертати структуровані дані, я б обрав temperature=0.0 та top_p=0.1, щоб зменшити випадковість і підвищити стабільність відповідей. Налаштування за замовчуванням можуть виявитися непридатними, оскільки модель здатна генерувати зайвий текст, порушувати формат JSON або допускати фактичні помилки. Тому параметри генерації потрібно підбирати відповідно до завдання та додатково перевіряти правильність отриманих даних.

Висновок за завданням 6

У ході виконання завдання було досліджено вплив параметрів temperature і top_p на різноманітність, точність та стабільність відповідей мовної моделі. Встановлено, що при temperature=0.0 модель генерувала більш передбачувані відповіді та правильно відповідала на фактичне питання у 5 випадках із 5. При підвищенні температури до 1.5 різноманітність зросла, проте кількість правильних відповідей зменшилася до 2 із 5, а в тексті з'явилися помилки та беззмістовні фрагменти. Зміна top_p також впливала на різноманітність генерації, але іншим механізмом. Отже, для роботи зі структурованими даними доцільно використовувати низькі значення temperature і перевіряти правильність результатів, а для творчих завдань — вищі значення, що забезпечують більшу різноманітність відповідей.

Завдання 8. Звести все разом і зробити висновок

Таблиця 18: Підсумкове порівняння моделей

Критерій	Локальна мала (llama3.2:3b)	Локальна більша (qwen3:4b)	Хмарна openai/gpt-

			oss-20b
Швидкість генерації	226,7 симв./с для довгого запиту	72,82 токенів/с у завданні 1	1674,7 симв./с для довгого запиту
TTFT	0,039–0,078 с	Не вимірювався	0,442–0,581 с
Якість, балів /12	3/12	10/12	10/12
Стабільність формату (завдання 2)	Правильний JSON у 2 із 2 запусків	Правильний JSON у 2 із 2 запусків	Правильний JSON у 2 із 2 запусків
Потрібні ресурси	Невеликі ресурси комп'ютера, RAM/CPU або GPU	Вищі вимоги до RAM/VRAM і обчислень	Інтернет, API-ключ, мінімальні локальні ресурси
Вартість	Безкоштовна модель, витрати на обладнання та електроенергію	Безкоштовна модель, витрати на обладнання та електроенергію	Залежить від тарифу та лімітів Groq
Куди йдуть ваші дані	Залишаються на локальному комп'ютері	Залишаються на локальному комп'ютері	Передаються на сервери Groq
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Для простих запитів, швидкого отримання коротких відповідей і роботи без інтернету, якщо	Для завдань, де потрібні кращі міркування, програмування та обробка конфіденційних	Для складніших завдан, великих текстів і якісних

	допустимі обмеження якості.	даних без передавання їх стороннім серверам.	українськом овних відповідей, коли є стабільний інтернет і дозволено передавати дані зовнішньом у провайдеру.

Загальні висновки

1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;

2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;

3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;

4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

5) Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій.